

cs312_a1

Suze van Adrichem

September 2026

General notes

- I did not use the modal cluster for this assignment because it was so backed up
- The default recipe reaches a final validation loss of 2.930 ± 0.0025 (mean and standard deviation over ten seeds; the handout quotes 2.926). All losses in this report are final validation losses unless stated otherwise.

Baseline reference: the default d8 run

Numbers worth knowing by heart. Default recipe: d8 (8 layers, width 512, 35M parameters), 600,000 sequences of 1024 tokens (614M tokens), batch 64, so 9,375 optimizer steps; AdamW with peak learning rate 0.003, 1% warmup (94 steps), linear decay to zero, weight decay 0.1, gradient clipping at 1.0. One run takes 11–12 minutes on an H100.

- **Final loss:** validation 2.930 ± 0.0025 over ten seeds (range 0.008); training loss about 2.88–2.90 at the end. Two identical deterministic runs agree to six digits.
- **Shape of the training loss curve:** starts at 8.8 (uniform over the 4096-token vocabulary); about 5.7 at step 50, 5.0 at step 100, 4.4 at step 200, 3.8 at step 400, 3.5 at step 800, 3.3 at step 2,000, 3.1 at step 5,000, then a bend down to 2.9 in the final decay. Fast until step 500, then slow: the loss drops 0.15 per thousand steps around step 1,000–2,000, 0.06 per thousand around step 3,000–5,000, and the linear decay adds a final 0.1 in the last quarter.
- **Per-step jitter:** the raw training loss scatters ± 0.04 (standard deviation) around its running mean at batch 64; that scales as $1/\sqrt{\text{batch}}$ and depends on nothing else.
- **Noise floor:** std 0.0025 across seeds; a difference below about 0.008 between two single runs is not meaningful. Sources: initialisation and data order each a few thousandths, GPU non-determinism about 0.001.

- **Internal statistics:** parameter RMS grows from 0.045 (attention) and 0.026 (MLP) at initialisation to about 0.09 mid-run and shrinks 10% in the decay; residual-stream RMS is 5–30 throughout (grows with depth, peaks near step 1,800); global gradient RMS 10^{-5} , i.e. a gradient norm of about 16 at step 0 (clipped) falling to about 0.14 at the end.
- **Sensitivities around the default:** a $3\times$ learning-rate change costs 0.02–0.05, $10\times$ costs 0.1–0.2; no warmup costs 0.1; never decaying costs 0.25; batch 256 costs 0.09; a wrong weight decay by $3\times$ costs 0.01–0.04.

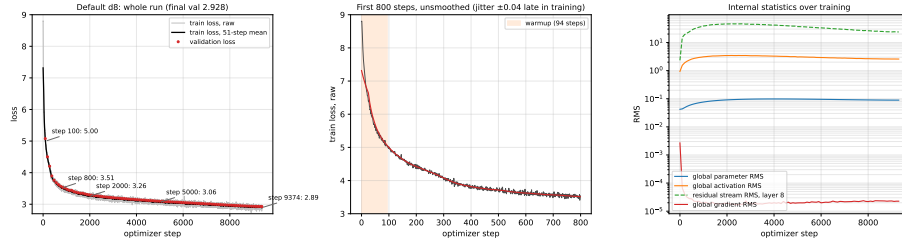


Figure 1: The default d8 run. Left: training loss (raw and 51-step mean) with validation points and landmark steps. Middle: the first 800 steps, unsmoothed, with the warmup shaded. Right: global parameter, activation and gradient RMS and the layer-8 residual-stream RMS over training.

1 Problem 1: Hyperparameters

1.1 (a)

I varied warmup, learning rate, batch size, and weight decay on their own. For each I chose the following variations:

Hyperparameter	Sweep values (default in bold, * = added after the first sweep)
Learning rate	0.0003, 0.001, 0.0015*, 0.002*, 0.003 , 0.0045*, 0.006*, 0.009, 0.027
Batch size	8*, 16, 32, 64 , 128, 256
Weight decay	0.011, 0.033, 0.1 , 0.3, 1.0
Warmup (fraction)	0, 0.003, 0.01 , 0.03, 0.1, 0.3*, 0.6*

Table 1: Problem 1(a) single-axis sweeps around the d8 defaults. The initial grid used a base-3 step (base 2 for batch size); the starred points were added afterwards to resolve the learning-rate optimum at factor-1.5 resolution and to find where the batch-size and warmup curves turn up. All runs use 600,000 training sequences (614M tokens), so batch size changes the number of optimizer steps, not the tokens seen.

Figure 2 shows the final validation loss along each axis; the default recipe reaches 2.9275. To judge which differences are real I use the run-to-run spread measured in Problem 3: repeating the default recipe with different seeds gives a standard deviation of about 0.002, so I treat differences below 0.005 between two single runs as noise.

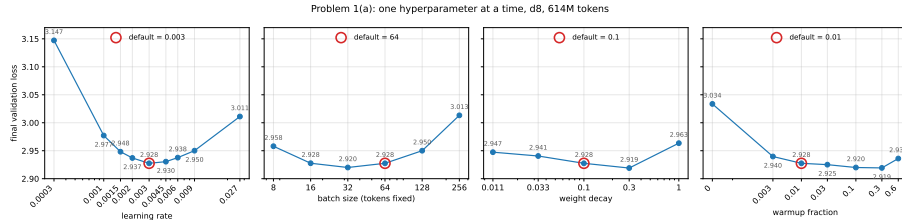


Figure 2: Varying individual hyperparameters from default d8 config

Learning rate is the most sensitive hyperparameter: the default 0.003 is already optimal and the loss is convex in $\log(\text{lr})$, with $3\times$ lower costing $+0.050$ and $3\times$ higher $+0.023$. Too low hurts more than too high ($10\times$ lower: $+0.22$). Batch size (at fixed tokens) is best at 32 (2.920, -0.008); 16 ties the default and larger batches degrade steeply (256: $+0.086$), because they take far fewer optimizer steps. The batch-32 gain is about 2σ from single runs, so it is likely but I would want more seeds to be sure. Weight decay does better at 0.3

than the default 0.1 (-0.009); both weaker decay and 1.0 are worse. Warmup improves monotonically with length: no warmup is very harmful ($+0.106$), and 10% warmup gains -0.008 (2.920); a further point at 30% (2.919) shows the gain saturates around 10%.

Ranking by how much the loss moves over the range tested: learning rate (0.22) > warmup (0.11) $\approx$ batch size (0.09) > weight decay (0.04). Valid ranges at this scale are roughly lr 0.001–0.01, batch 16–64, weight decay 0.1–0.3, warmup $\geq 1\%$.

For a grid search, I would search in log space with a base-3 step and about 5 points per axis, centred on the d8 defaults which the smoke run had already shown to train cleanly. Learning rate should be tuned first since it dominates. Warmup and batch size come next, since they moved the loss the most after learning rate, and I would cross them with learning rate rather than sweep them alone, because the best batch size or warmup may depend on the learning rate. Weight decay matters least and can be tuned last over a narrow range. Any winner by less than 0.01 should be re-run with a few seeds before trusting it.

Takeaways. Learning rate is the knob that matters most and the default is near its optimum; too low hurts more than too high. Warmup is the second most important thing and more of it (up to about 10%) is free. Batch size has a shallow optimum at fixed tokens; going far above the default starves the run of steps. Weight decay is a second-order knob.

1.2 (b)

I paired the learning rate with batch size, weight decay and warmup, one small grid each around the defaults with everything else at the default (figure 3; 35 single-seed runs including follow-ups).

For **batch size**, the best learning rate is 0.003 at every batch on the coarse grid, but a finer grid at batch 16 shows its optimum is about half that, so the two co-vary at small batch, roughly $\text{lr} \propto \sqrt{\text{batch}}$. From batch 32 up the optimum stays put, and above batch 64 no learning rate helps: at fixed tokens those runs have too few steps. For **weight decay**, the best value drops one grid step for every step up in learning rate, a diagonal across the panel: AdamW decays by $\text{lr} \cdot \text{wd}$ per step and the best product is about 10^{-3} at every learning rate. For **warmup**, the best learning rate rises with warmup length; with 10% warmup it sits near 0.009 rather than 0.003, and lr 0.009 with 10% warmup is the best run in Problem 1 at 2.914, 0.014 below the default. The optimal warmup itself stays around 10% at every learning rate; what grows with the learning rate is the cost of too little.

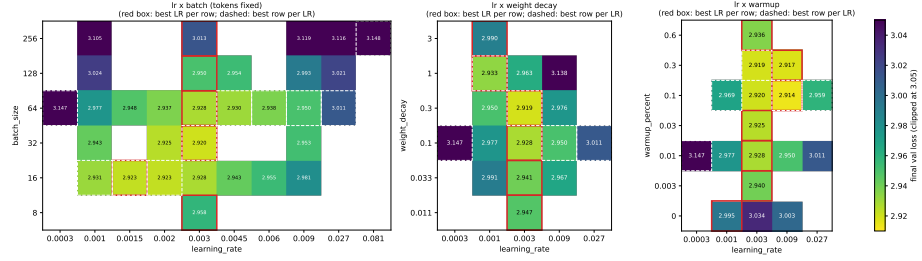


Figure 3: Final validation loss for pairs of hyperparameters, all other settings at the d8 defaults. Red boxes mark the best learning rate in each row; dashed boxes the best row value in each column.

Takeaways. Hyperparameters that must be tuned together: learning rate with warmup (a higher learning rate needs and rewards a longer warmup), learning rate with weight decay (keep their product roughly fixed), and learning rate with batch size only at small batches (roughly $\text{lr} \propto \sqrt{\text{batch}}$). Improvements found one at a time do not add up. A coarse grid can hide a shift of the optimum smaller than its step; the tilt of the curve gives it away.

1.3 (c)

My goal here was to figure out how learning rate schedulers interact with the other hyperparameters. I mostly focus on the learning rate since that is what the scheduler acts on directly. My hypothesis was that different schedulers would have different optimal learning rates: a scheduler that spends longer at the peak learning rate should prefer a lower peak.

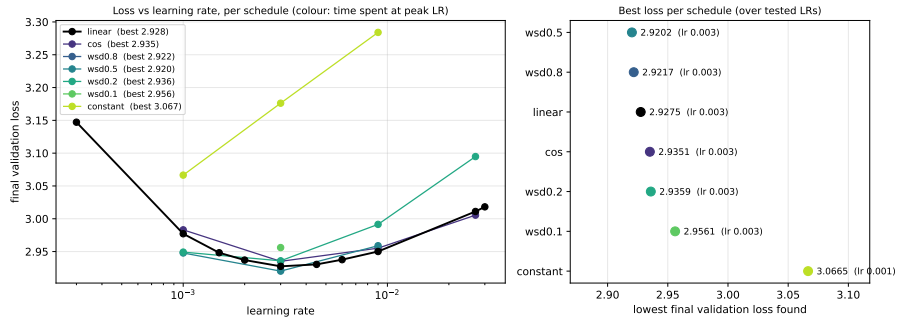


Figure 4: Left: final validation loss against learning rate, one line per scheduler (all other settings at the d8 defaults). Right: the lowest loss found for each scheduler, with the learning rate that achieved it.

As can be seen in figure 4, constant is by far the worst scheduler at every learning rate. The rest are much closer. At the default learning rate wsd0.5

does best, slightly ahead of linear, while a short decay (wsd0.1, wsd0.2) is clearly worse: the decay fraction itself turned out to be a hyperparameter worth tuning, with about half the run spent decaying being the sweet spot. As the learning rate rises, the schedulers that hold the peak longer (wsd0.2, constant) fall behind faster than linear and cosine do, which is the direction my hypothesis predicted. Completing the left side of each bowl with runs at lr 0.001 confirmed the direction but not a moved optimum: every decaying scheduler still has its minimum at 0.003 on this grid, but the longer a scheduler holds the peak, the less it loses from going lower and the more from going higher, so their true optima sit somewhere between 0.001 and 0.003. Constant is the extreme case and prefers a learning rate below 0.001.

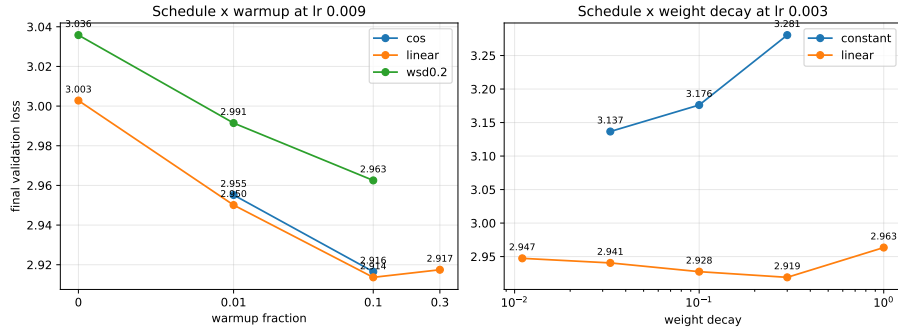


Figure 5: Left: warmup fraction against final validation loss for three schedulers at lr 0.009. Right: weight decay against final validation loss for the linear and constant schedulers at lr 0.003.

I also did two brief experiments crossing the scheduler with warmup and with weight decay (figure 5). For warmup, all schedulers are affected about equally: at lr 0.009, going from no warmup to 1% to 10% lowers the loss by a similar amount for linear, cosine and wsd0.2. Warmup acts on the start of training and the decay on the end, and the two effects simply add. For weight decay, the optimum depends on the scheduler. Under linear decay the best weight decay is 0.3, but under the constant scheduler it drops to 0.033 or below. This matches the product rule from part (b): what matters is the learning rate times the weight decay integrated over the run, and a scheduler that never decays applies more total decay for the same setting, so it wants a smaller value. One caveat is that the constant scheduler in this codebase also skips warmup, so its runs are penalised twice; comparing wsd0.5 against linear would be the cleaner version of this test.

I did not have the time to check the interaction of any other hyperparameters.

Takeaways. Always decay the learning rate; how you decay matters far less than whether you do, and about half the run spent decaying is the sweet spot.

Schedules that hold the peak longer prefer a lower peak. Warmup acts on the start and the schedule on the end, so their effects add.

2 Problem 2: Predicting the reliability of a scaling law

2.1 (a)

I trained four variants of the d8 recipe (baseline, constant learning rate, dropout 0.2, learning rate 0.03) at depths d4 to d9. Width grows with depth along this ladder (hidden size is 64 times the depth). Following the handout, the x-axis is compute relative to d8, which at a fixed token budget is $N/N_{d8} = (\text{depth}/8)^3$, and the fit is $L = a C^{-\alpha} + \varepsilon$ with ε the irreducible loss the curve flattens toward.

I fit on d4–d7 first and wrote down predictions for d8 and d9 before training them (figure 6, left). They held: the baseline’s held-out points landed on its curve almost exactly, the others within about 0.01, and the ordering was right (worst to best: constant lr, dropout 0.2, lr 0.03, baseline). Baseline and dropout follow clean power laws; constant and lr 0.03 bend, their gap to the baseline widening with size, because a fixed learning rate or a missing decay hurts a bigger model more. Fitting on all six points and extrapolating to d20 (right) gives about 2.67 for the baseline, in a range of 2.63–2.70 over fits the data cannot distinguish; the constant-lr range is wider because its floor is poorly pinned, and the dropout fit found no floor at all, so its extrapolation is the least trustworthy despite looking straightest.

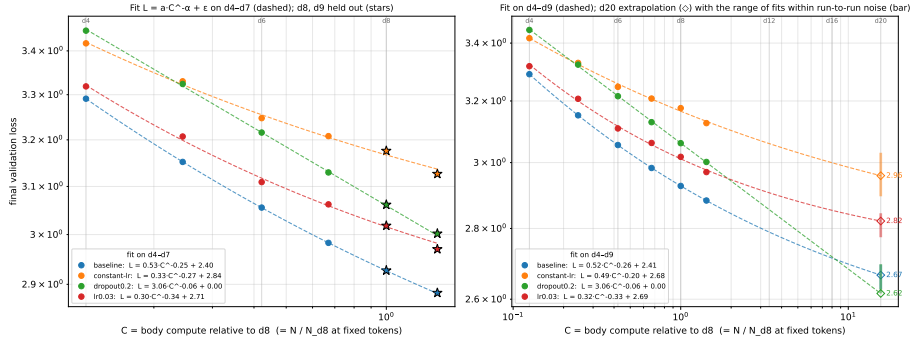


Figure 6: Scaling ladders for four variants of the d8 recipe, d4–d9, with width growing with depth; x-axis is compute relative to d8. Left: fits on d4–d7 (dashed) with d8 and d9 held out (stars). Right: fits on all points, extrapolated to d20 (diamonds) with the range of fits compatible with run-to-run noise (bars).

Takeaways. A recipe that stays equally good at every size gives a clean power law that predicts one step ahead to within a few thousandths. A fixed hyperparameter whose optimum moves with scale (learning rate, schedule) bends the law, always toward higher loss for the big models. Far extrapolation hinges on the irreducible term, which a handful of small runs cannot pin down, so quote a range.

2.2 (b)

I looked for changes to d8 that keep a power law but change its slope. Weight decay 0.3 gave exactly the same slope as the baseline, just a small constant offset (figure 7). Lowering the learning rate to 0.001 turned out to be a near-parallel shift too, about 0.05 above the baseline at every size with only a hint of convergence at d9. So neither weight decay nor a lower learning rate changed the slope on the model axis; the slope changes come from the constant-lr and lr 0.03 ladders in (a).

I also scaled data instead of mWait, Figure 9 is still two panels, right? odel size, keeping d8 fixed and varying the tokens (figure 8). This bends rather than staying a straight line: the slope keeps falling, and the fitted floor is higher than on the model axis. More data alone stops helping sooner than more model alone.

On the data axis I also tried a higher learning rate, 0.009, along the token ladder. This is the clearest slope change I found: it costs almost half a nat at 77M tokens and almost nothing at 1.23B, so the curve is much steeper than the baseline’s while staying smooth. Warmup and decay are fractions of the run, so a short run at a hot learning rate gets a dozen warmup steps and a very short anneal while a long run gets hundreds and thousands. Anything defined as a fraction of the run changes how an intervention acts along a data ladder.

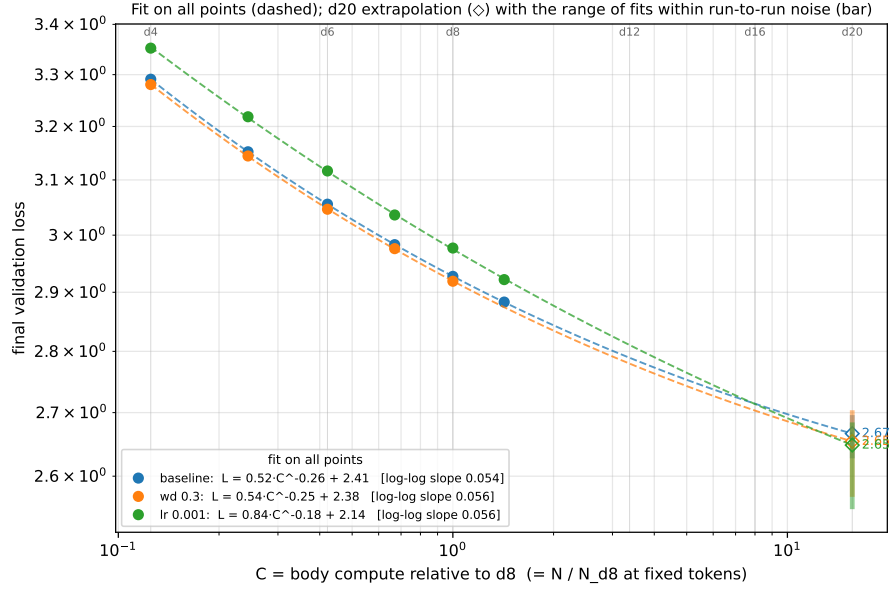


Figure 7: Weight decay 0.3 and lr 0.001 ladders against the baseline, same conventions as figure 6.

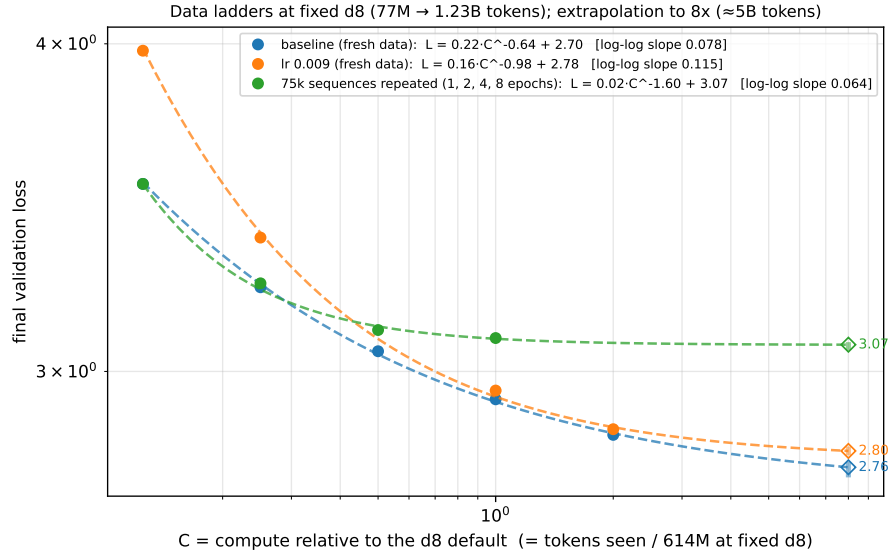


Figure 8: d8 trained on 77M to 1.23B tokens; x-axis is tokens relative to the 614M default.

Takeaways. Interventions that add a constant cost or benefit (dropout, weight decay, step starvation) shift the law in parallel and are safe to extrapolate. Slope changes come from hyperparameters whose optimum depends on scale, and, along a data ladder, from anything defined as a fraction of the run (the warmup and the learning-rate decay phase), because a short run gets only a few steps of each. Scaling data alone at fixed model size bends toward a floor; scaling either axis alone runs out.

2.3 (c)

Here I tried to break the power law outright. I picked interventions that should hurt the large models more than the small ones, since that is what makes a fit on small models mislead. Each is a d4–d8 ladder; the test is to fit on d4–d6 and see whether d8 lands where the fit says (figure 9).

Two broke. Training every model on one eighth of the data stopped improving at d6: past that point the model is limited by data, not size, and the small-model fit predicts a d8 that never arrives. Plain SGD never learned more than token frequencies and got slightly worse with size, so there is no law to fit at all. Two did not break. Batch 256 is a constant penalty at every size, so the fit still holds. A hot recipe (lr 0.03, no warmup, no clipping) bends away from the baseline as the model grows but stays smooth, with no spikes or divergence at these sizes.

On the data axis, the break comes from repetition. Keeping d8 and 75k unique sequences fixed and scaling the tokens seen by repeating the data for 2, 4 and 8 epochs looks like a data ladder on the compute axis, but the second pass is nearly as good as fresh data, the fourth is clearly behind, and from four to eight epochs the loss barely moves while fresh data keeps improving. The law in tokens seen holds for about two passes and then flattens: the compute is buying re-reads, not information. The axis being scaled has again stopped being the binding constraint.

What to look for in a pilot plot. From (a)–(c), a naive projection is trustworthy when the recipe stays equally good at every size: fixed hyperparameters that are near their optimum, fresh data with tokens growing alongside the model, an optimizer that normalises per parameter. It bends when a fixed hyperparameter has an optimum that moves with scale (learning rate, schedule, warmup): the small models look fine and the large ones drift off the line, always in the direction of higher loss. It breaks when the axis being scaled is not the binding constraint (more model on too little data, or more passes over the same data), or when the optimizer cannot use added capacity. So the questions to ask of a pilot are: is the data budget growing with the model, is anything held fixed that should scale, and does the optimizer adapt per parameter? If the answer to any is no, expect the projection to be optimistic.

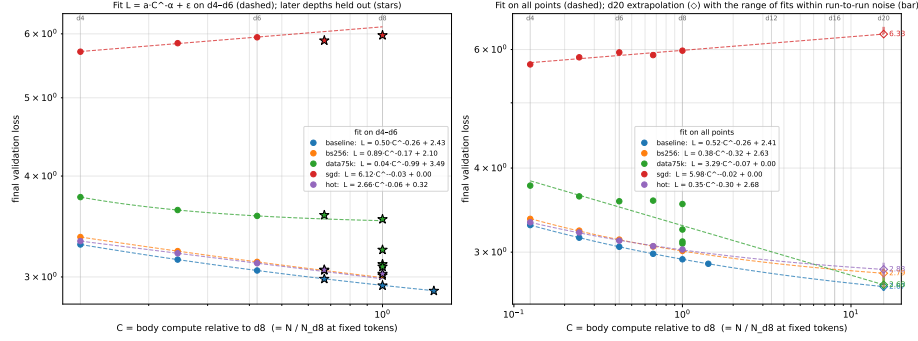


Figure 9: Four ladders designed to break the law, against the baseline; same conventions as figure 6, with d8 held out on the left.

Takeaways. A scaling law breaks when the thing being scaled is not the binding constraint (more model on too little data, more passes over the same data) or when the optimizer cannot use added capacity (plain SGD). Merely sub-optimal hyperparameters bend but do not break. Before trusting a pilot, ask whether the data grows with the model, whether anything is held fixed that should scale, and whether the pilot regime has an artifact that the endpoint will not share.

3 Problem 3: Measuring variation

3.1 (a)

I ran the default d8 recipe ten times with a different model seed and data seed each time and deterministic kernels off, so initialisation, data order and GPU non-determinism all vary (three seeds at first, seven more added once it was clear that three cannot pin down a standard deviation). Figure 10 compares them to the deterministic reference run.

The terminal losses span about 0.008, with a standard deviation of 0.0025, so a difference below about 0.008 between two single runs is noise. The distribution is roughly symmetric around 2.930 with no outlier; the deterministic reference sits in its lower third. The smoothed training curves are indistinguishable, so the randomness does not change the shape of the curve, only where it ends up. What it does change is the step-to-step jitter: different data seeds mean different batches at each step, and the raw per-step losses of the three seeds are unrelated to each other, whereas runs sharing a data seed (as in Problem 1) have identical jitter.

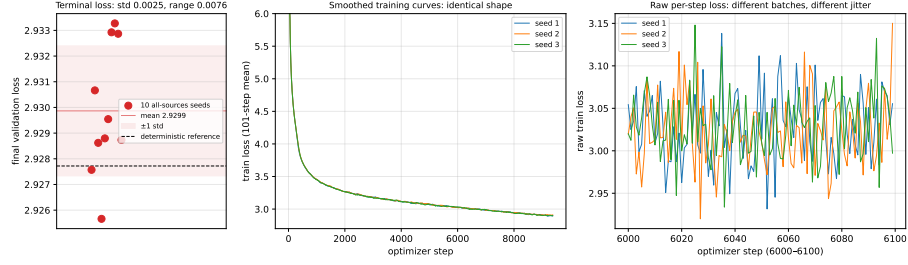


Figure 10: Ten all-sources seeds of the default recipe. Left: terminal validation loss against the deterministic reference, with mean and one standard deviation. Middle: smoothed training curves of three seeds. Right: raw per-step training loss of the same three over 100 steps.

Takeaways. For the default recipe, two runs differ by a few thousandths for no reason at all; treat anything below about 0.008 as noise. Randomness changes the level of the curve and its per-step texture, not its shape.

3.2 (b)

I isolated each source using deterministic mode: three runs varying only the model seed, three varying only the data seed, three identical runs with deterministic kernels off, and the reference config rerun on an A100 (figure 11). Each source alone produces a spread of a few thousandths, and varying all three together gives no more spread than any one of them. The sources are not additive: any perturbation puts the run at a random point within the same noise floor, which matches the chaotic picture from lecture and the one-token result in Problem 4. The apparent ordering (initialisation largest, kernels smallest) is within what three samples can resolve. The GPU is the smallest source: switching from H100 to A100 changes the loss by less than kernel noise on a single GPU.



Figure 11: Final validation loss with one source of randomness varied at a time (left) and the standard deviation per source (right).

Takeaways. The sources of randomness are not additive: any perturbation, however small, lands the run somewhere in the same noise band. The GPU is not a meaningful source. Deterministic mode buys exact reproducibility, not a smaller noise floor.

3.3 (c)

I repeated the seed experiment from (a), three all-sources seeds per setting, for several changes to the recipe and compared the spread with the baseline’s (figure 12). Two changes clearly widen the noise floor: removing warmup makes it about ten times larger, because the unstable first steps send each seed on a visibly different path, and the small d4 model is several times noisier than d8. For the other settings, a $3\times$ higher learning rate, batch 16, d6 and training on twice the tokens, the measured spreads differ from the baseline’s by factors of one to a few, which three seeds per setting cannot resolve; the staff’s ten-seed measurement puts the learning-rate effect at about $1.6\times$.

For more capable models the answer is that a lower loss does not buy a lower noise floor: the d8 model trained on twice the tokens reached a clearly lower loss but had the same run-to-run spread as the baseline. Across model size there is no smooth trend either; only the smallest model is clearly less reproducible.

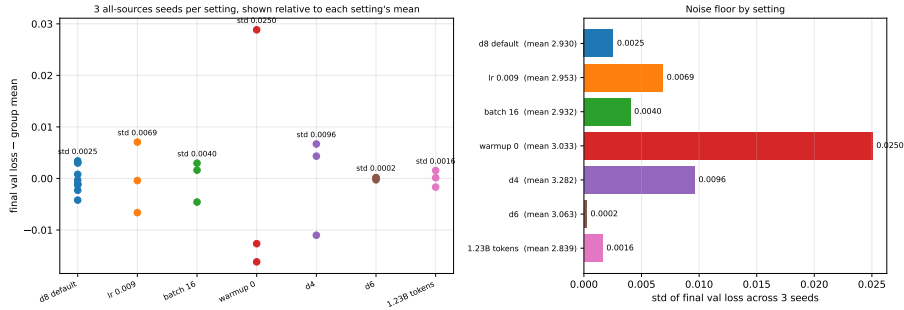


Figure 12: Three all-sources seeds per setting, shown relative to each setting’s mean (left), and the resulting standard deviation (right).

Takeaways. The noise floor belongs to the recipe, not the model: hotter or noisier optimization widens it, and a lower loss does not narrow it. Moderate changes (a $3\times$ learning rate, a $4\times$ smaller batch) shift the floor by tens of percent; only destabilising changes such as removing warmup multiply it, by about tenfold. Re-measure the floor whenever you leave the default recipe, and remember that a standard deviation from three seeds is uncertain by half.

4 Problem 4: Amplification of randomness

4.1 (a)

I compared two deterministic d8 runs that are identical except that one token in one training sequence is changed (figure 13). The terminal validation loss differs by 0.0023 and the terminal training loss (mean of the last 100 steps) by 0.0018. That is as large as the spread from a completely different initialisation in Problem 3, from a change of one token in 614 million. The difference appears at the very first evaluation, where it is largest, and then settles to the size of ordinary run-to-run noise for the rest of training. So training does not slowly amplify the perturbation; it immediately puts the run on a different trajectory, after which the two runs are simply two different draws from the noise distribution.

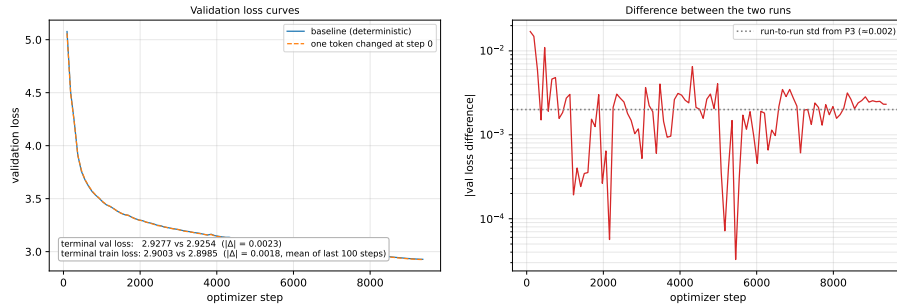


Figure 13: Baseline vs. one token changed at step 0, both deterministic. Left: validation loss curves with terminal losses. Right: absolute difference between the two runs over training; the dotted line is the run-to-run standard deviation from Problem 3.

Takeaways. Training is chaotic in its first few hundred steps: a single changed token is amplified to the full run-to-run noise immediately, and after that the two runs are just two draws from the noise distribution.

4.2 (b)

I then varied when and how much of the data is changed, keeping everything deterministic so the only difference from the baseline is the perturbation (figure 14). For timing I changed one token in the batch at step 0, 1000, 3000, 6000 or 9000. For size I changed 1, 10, 100 or 924 tokens in one sequence, or all 924 tokens in all 64 sequences of a batch, at step 0 and again at step 6000.

How many tokens are changed makes almost no difference to the final loss: one token and a whole batch end within the same noise band, and the only trace of the size is a somewhat larger spike right after the change. When the change happens makes all the difference. A change in the first few hundred steps is amplified to the full run-to-run noise, the same 0.002 as in (a); a change at

step 1000 leaves a final difference fifty times smaller, and by step 6000 even a whole batch is forgotten to within 0.00001. Both the initial spike and the final difference fall steadily the later the change is made. Early training is the chaotic phase where trajectories diverge; once the learning rate has decayed, training damps perturbations instead of amplifying them.

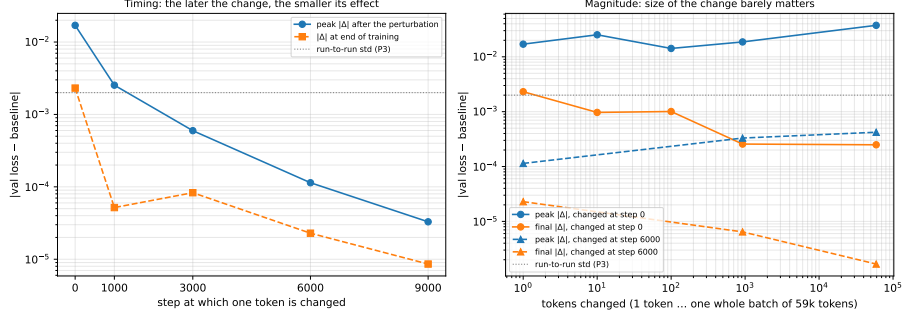


Figure 14: Effect of a perturbation on the validation loss, measured as the peak difference from the baseline after the change and the difference at the end of training. Left: one token changed at different steps. Right: different numbers of tokens changed at step 0 (solid) and step 6000 (dashed). The dotted line is the run-to-run standard deviation from Problem 3.

Takeaways. When a perturbation happens matters; how big it is does not. Anything in the first few hundred steps is amplified to the noise floor; anything after step 1,000 is forgotten. Late training is a contraction, not an amplifier.

5 Problem 5: Loss curve augury

5.1 (a)

Problems 1–3 already gave training loss curves for the learning rate, batch size, warmup and schedule, so the only new runs were a momentum sweep ($\beta_1 = 0, 0.8, 0.98$ against the default 0.9). Figure 15 shows the training loss (smoothed with a 101-step running mean) for each knob.

Every curve has the same template: a fast drop in the first hundred steps, a long slow descent, and a bend downward at the end as the learning rate decays to zero. The hyperparameters change this template as follows.

- **Learning rate** changes both the height and the shape. A low learning rate descends late and slowly and never catches up; a high one descends sharply early, then flattens and is overtaken. The curves cross.
- **Batch size** mostly changes the height. The runs also end at different step counts (batch 256 stops at 2,344 steps, batch 16 at 37,500), so on a shared step axis the final bend happens at a different place for each.

- **Momentum** matters mainly in the first thousand steps. No momentum is slower from the first steps and stays behind; $\beta_1 = 0.98$ lags early, because the running average takes hundreds of steps to build, and only partly recovers. 0.8 and 0.9 are indistinguishable.
- **Schedule** leaves the curve identical until the decay begins, after which the curves fan out; only the very end changes.
- **Warmup** matters at the beginning. No warmup is actually the lowest curve for the first fifty steps, since it takes full-size steps at once, and the highest curve for the rest of the run. Longer warmups start slower and cross below the default around step 1,000.

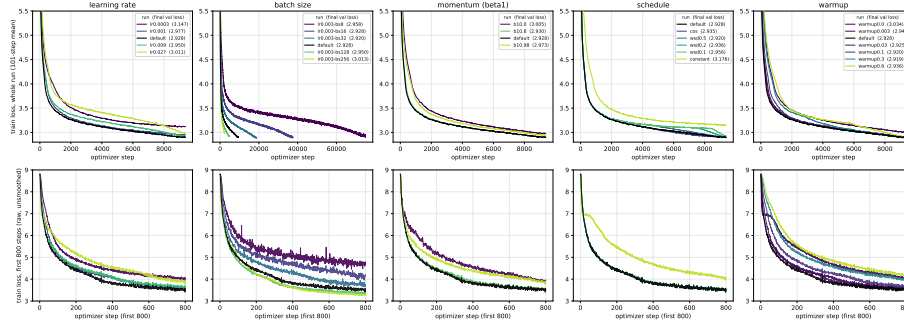


Figure 15: Training loss curves, one column per hyperparameter, coloured by the value of the varied hyperparameter with the default in black and the final validation loss in the legend. Top: whole run, 101-step running mean. Bottom: first 800 steps, raw per-step loss.

Takeaways. Every healthy curve has the same template: fast drop, slow descent, final bend. Learning rate sets the level and the crossing of curves, warmup and momentum shape the first thousand steps, the schedule shapes only the end. A too-high learning rate shows as an earlier, higher plateau, not as more noise.

5.2 (b)

I measure smoothness as the standard deviation of the raw per-step loss around its 51-step running mean, computed over the second half of training where the curve is nearly flat (figure 16). Batch size is the only ordinary hyperparameter that changes it: the jitter falls exactly as one over the square root of the batch, from 0.08 at batch 8 to 0.02 at batch 256. Learning rate, momentum, schedule, warmup, weight decay, dropout, gradient clipping and QK-norm all leave it at the default 0.04. The reason is in the right panel: runs that share the data seed have per-step jitter that is correlated 0.98–1.0 with the default run’s, point for point, whichever knob was changed, while a different data seed gives zero

correlation at the same size. The jitter is the difficulty of each batch, not optimizer noise, and only how many examples are averaged per step changes its size. This holds from the start of training as well: measured in windows from step 100 onwards, every learning rate has the same jitter in every window (about 0.03 early, rising to 0.04), while batch 256 sits at half that throughout and batch 16 at double. A higher learning rate does not make the early curve noisier in these runs.

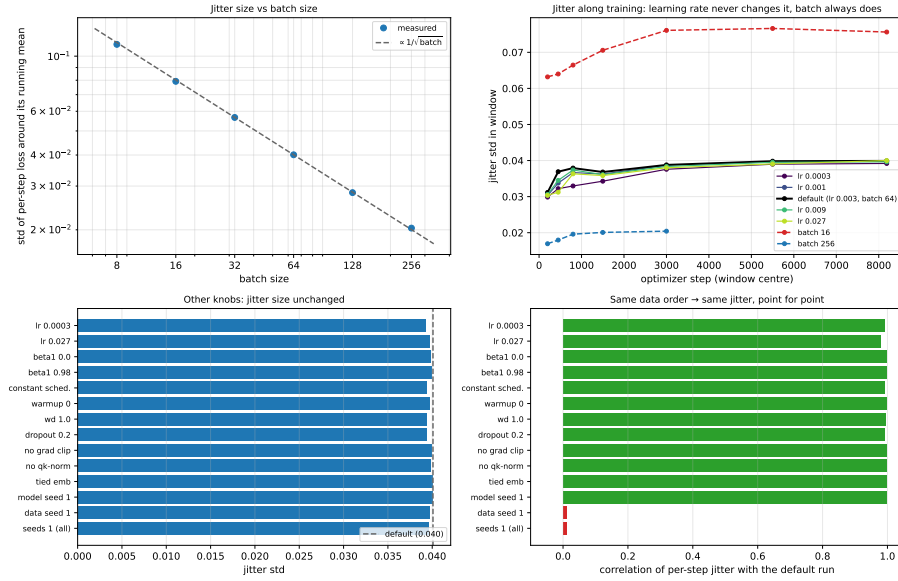


Figure 16: Micro-structure of the training loss. Top left: jitter size against batch size with a $1/\sqrt{\text{batch}}$ line. Top right: jitter measured in windows along training for the learning-rate sweep (colour gradient, default in black) and two batch sizes (dashed). Bottom left: jitter size for other changes to the recipe, all at the default level. Bottom right: correlation of each run’s per-step jitter with the default run’s; green runs share the default data order, red ones do not.

Takeaways. The per-step jitter is the difficulty of each batch: its size is set by the batch size alone, as $1/\sqrt{\text{batch}}$, and runs sharing a data order share the jitter point for point.

5.3 (c)

Figure 17 collects the runs from this assignment whose curves look different from the default. On the macro scale the standouts are plain SGD, which climbs to a loss of 7 and never learns more than token frequencies, the constant schedule, which lacks the final bend, and no momentum, which is slower throughout. Micro-scale changes are invisible by eye, so I counted spikes: steps where the

loss exceeds its running mean by more than five standard deviations. Every healthy run has at most two, of about 0.2 nats. SGD has 186 reaching 9 nats, and weight decay 1.0 at learning rate 0.009 has 21 reaching 0.6, so these two runs are unstable throughout. The recipe with learning rate 0.03, no warmup and no clipping is the opposite case: it overshoots to more than twice the initial loss in the first steps but is perfectly normal afterwards, so its damage is done early.

Good training here looks like a fast initial drop with no overshoot, a smooth descent with the batch-size jitter from (b) and no isolated spikes, and a clear downward bend at the end. Bad training shows up as an early overshoot (no warmup, hot learning rate), a missing final bend (no decay), a slow start that never recovers (no momentum, learning rate too low), or spikes far above the jitter level (unstable optimizer, over-regularisation).

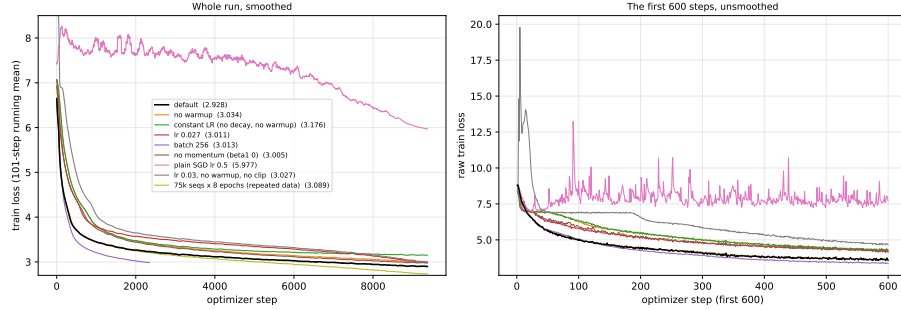


Figure 17: Runs whose training loss curves differ from the default. Left: smoothed curves over the whole run. Right: raw per-step loss over the first 600 steps.

Takeaways. Healthy: no early overshoot, smooth descent with the batch-size jitter, clear final bend. Unhealthy: an early overshoot (no warmup, hot learning rate), no final bend (no decay), a slow start that never recovers (no momentum, too-low learning rate), or spikes far above the jitter (unstable optimizer, over-regularisation).

6 Problem 6: Activations and gradient norms

6.1 (a)

For the default d8 run I looked at the RMS of the parameters, activations and gradients of each layer’s modules at the start (step 100), middle and end of training (figure 24). The RMS is the square root of the mean of the squared entries of a tensor, so it is the typical size of an entry and does not depend on how many entries there are. The L2 norm is the square root of the sum of squares, so it equals the RMS times the square root of the number of entries; the gradient

norm that clipping acts on is the L2 norm of all gradients together, which for d8 is about 5,900 times the global gradient RMS. Only the RMS is logged, so I did not look at the max or quantiles. I would expect the parameters to stay roughly Gaussian, with a max of about five times the RMS, while activations and gradients are heavy-tailed: a few large channels in the deeper layers’ residual stream, and rare, large gradients on infrequent token embeddings and in the first layer at the start of training. A max detaching from the RMS would be the early sign of instability. The figure shows five of the fourteen per-layer modules: the query and output projections of attention, the up and down projections of the MLP, and the input RMSNorm. The key and value projections behave like the query projection, the gate projection like the up projection, and the post-attention norm like the input norm, so they are left out.

Parameters. The RMS of the parameters is simply how big the weights are: the typical size of an entry in that module’s weight matrix. Regardless of layer or operation, the weights start small (the initialisation scale), grow two to three times by the middle of training, and then shrink a little by the end, when the learning rate has decayed and weight decay wins. The projections that write into the residual stream, the attention output projection and the MLP down projection, end larger in deeper layers, while the query projection is flat across depth.

Activations. The RMS of the activations is the scale of the signal flowing through that point of the forward pass. At the start it is roughly the same in every layer, because every layer starts at the same weight scale; the depth structure is learned. By the middle of training the activations have grown, and they grow with depth: both the reading projections (query, up) and the writing projections (output, down) are several times larger in layer 8 than in layer 1. This follows the residual stream itself, which each block adds to. In the trained network the stream grows faster than linearly with depth, roughly a constant factor of 1.25 per layer, i.e. close to exponential (from about 8 in the first layer to about 38 in the last at mid-training, a straight line on the log axis of figure 22); at initialisation the growth is closer to linear. Deeper blocks add proportionally more because their writing projections are larger. The middle values are higher than the final ones because activations follow the weights, which shrink during the decay phase. The input RMSNorm panel shows where the depth profile is set: its parameter is a learned gain (initialised to 1) and its activation is the normalised residual stream times that gain, i.e. what the block reads. By mid-training the gains have been turned down to 0.5–0.8 in the first two layers and up to 1.3–1.5 in the last four.

Gradients. The RMS of the gradients is how strongly each module is being pushed to change. At the start the push is concentrated in the first layer, about ten times larger than anywhere else regardless of the operation, and falls with depth: the network is learning token statistics first, while the gradient reaching later layers is still small because their inputs are random. By the middle of training this has flipped to a flat or slightly rising profile with depth, and the first layer’s gradient has dropped by an order of magnitude. The gradients at the end are somewhat larger than in the middle even though the weights

are smaller, and that is not a coincidence: every weight matrix here feeds a normalisation, so scaling the weights down leaves the function unchanged and scales the gradient up by the same factor. Gradient size goes as one over weight size. Smaller weights in the decay phase therefore mean larger gradients, the same inverse relation seen across runs in (b). Note the scale: gradient RMS is around 10^{-5} against parameter RMS around 10^{-1} , so the raw gradient does not set how far the weights move; Adam divides it out.

Takeaways. Weights grow then shrink slightly; activations follow the weights and grow superlinearly with depth (about 1.25x per layer) as the residual stream accumulates; gradients go as one over weight size and are front-loaded in the first layer early on. A residual stream in the tens is normal; in the thousands or beyond is a broken run.

6.2 (b)

I looked at how the sweeps from Problem 1 change the global parameter, activation and gradient RMS over training, one figure per sweep, with the curves coloured by the swept value and the default in black.

Learning rate (figure 18). A higher learning rate gives bigger weights throughout training, roughly as the square root of the learning rate over the 90x range tested, and the activations follow the weights up. The gradients go the other way: a higher learning rate gives smaller gradients, which is the one-over-weight-size relation from (a). A single learning-rate change moves all three statistics, weights and activations together and gradients opposite.

Weight decay (figure 19) is the clearest of the three sweeps. A large weight decay gives small parameters, which is what weight decay is for, and the activations are small with them; the gradients are the opposite, large when the weights are small. As with the learning rate, the values stay different all the way to the end of training. Weight decay 0.3 lands almost exactly where learning rate 0.001 does on all three statistics, so what sets the weight scale is the ratio of learning rate to weight decay, the same pairing that showed up as the product rule in Problem 1.

Warmup (figure 20). A longer warmup holds the learning rate small for longer, so the weights start growing later. The activations rise more slowly and their mid-training hump, the peak before the decay pulls them back down, moves later. The gradients stay high for longer, because they are large while the weights are still small. Everything ends at the same point regardless of warmup, since the decay phase that sets the final size is unchanged.

The important distinction between the sweeps is where the curves end up. Different learning rates (or weight decays) give weights, activations and gradients that are very different at the end of training: the change is permanent. Different warmups give curves that differ early and then converge, so at the end the trained networks look the same. Learning rate and weight decay set the scale of the network throughout training; warmup only changes how it gets there.

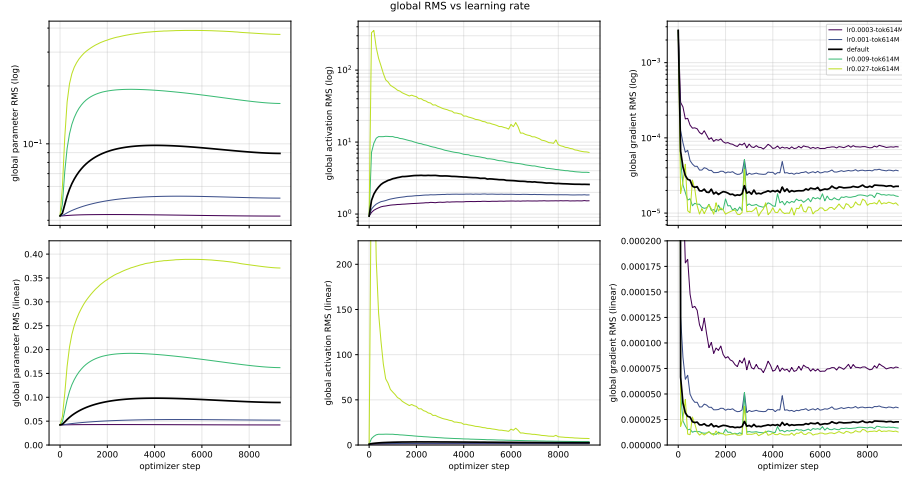


Figure 18: Global parameter, activation and gradient RMS over training for the learning-rate sweep. Top row: log y axis; bottom row: the same curves on a linear y axis, with the range set by the values after step 300 so the initial spike does not hide the trend. Colour follows the learning rate; the default is black.

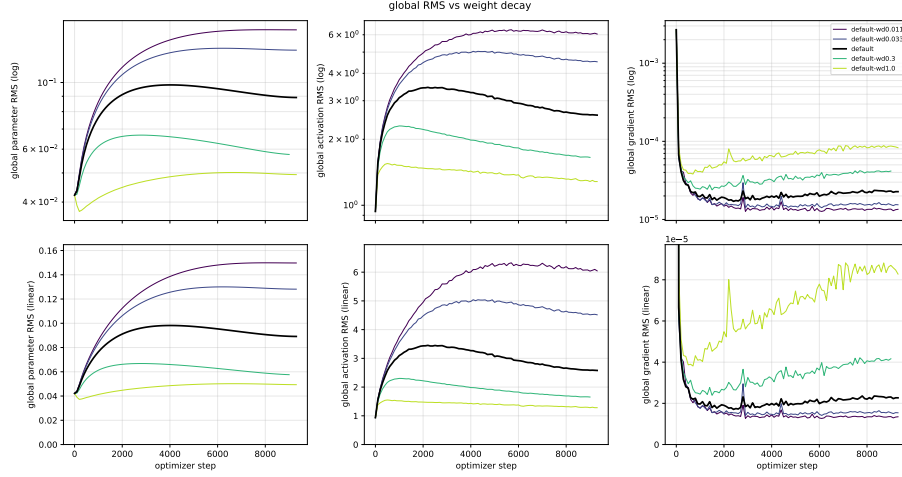


Figure 19: The same for the weight-decay sweep.

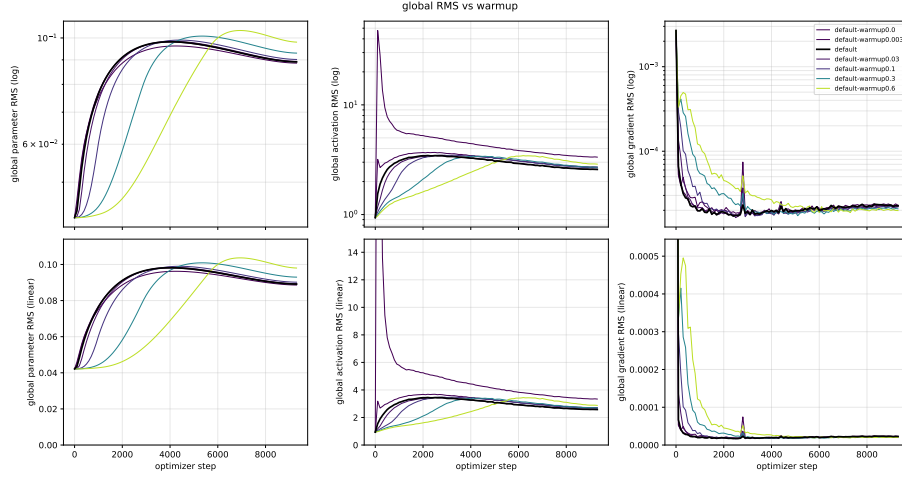


Figure 20: The same for the warmup sweep.

Takeaways. A higher learning rate or a lower weight decay makes the weights and activations bigger and the gradients smaller, for the whole run; their ratio is what sets the scale. Warmup only changes the path to the same end point.

6.3 (c)

Learning rate, weight decay and warmup were covered in (b). For (c) I added three runs that change the network or the update rule rather than the optimizer settings (figure 21): turning off gradient clipping (by default the gradient is rescaled to L2 norm 1 whenever it is larger); removing QK-norm (the RMSNorm applied to queries and keys inside attention before the dot product); and tying the embeddings (using one shared matrix for the input embedding and the output head instead of two). Putting these together with (b), the levers are:

- **If you want larger parameters and activations for the whole run:** raise the learning rate or lower the weight decay. The weight scale follows the ratio of the two, so either works.
- **If you want smaller parameters and activations for the whole run:** lower the learning rate or raise the weight decay.
- **If you want larger gradients:** make the weights smaller, so lower the learning rate or raise the weight decay; gradient size goes as one over weight size. If you only want them larger at the very start, turn off gradient clipping: the first step is otherwise clipped sixteen-fold, and by step 200 clipped and unclipped runs agree.

- **If you want smaller gradients:** make the weights larger, so raise the learning rate or lower the weight decay. If you only want the initial gradient smaller, tie the embeddings, which cuts it seven-fold at step 0 with no effect later.
- **If you want to change things only at the start of training:** change the warmup. No warmup gives a thirty-fold activation overshoot at step 100 that decays away; a longer warmup delays the growth of the weights and keeps the gradients high for longer. Clipping and tied embeddings, as above, also act only at the start. None of these leaves a trace at the end.
- **If you want to change things only at the end of training:** remove QK-norm to lower the final activation scale by about a fifth while leaving the weights unchanged, the one lever that moves activations without moving weights. Or lengthen the decay phase of the schedule, which shrinks the weights and raises the gradients in the final part of every run.

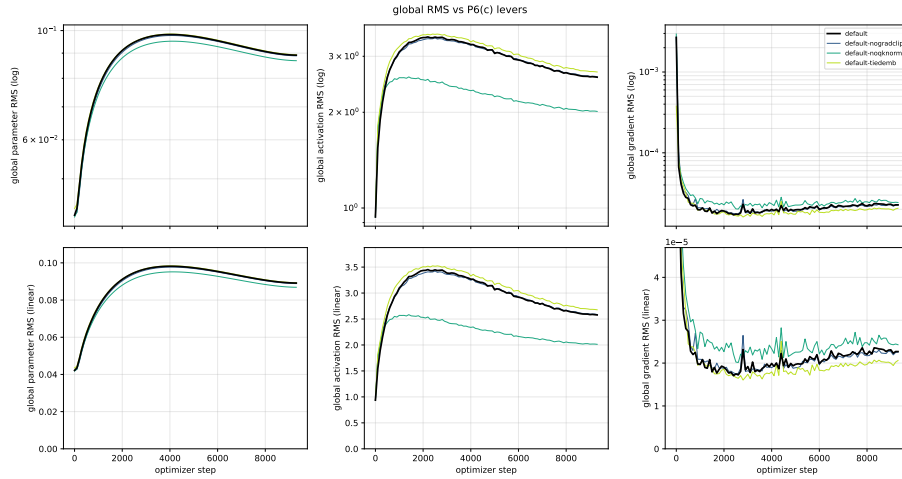


Figure 21: Global parameter, activation and gradient RMS over training for the default (black) and the three (c) changes: no gradient clipping, no QK-norm, tied embeddings.

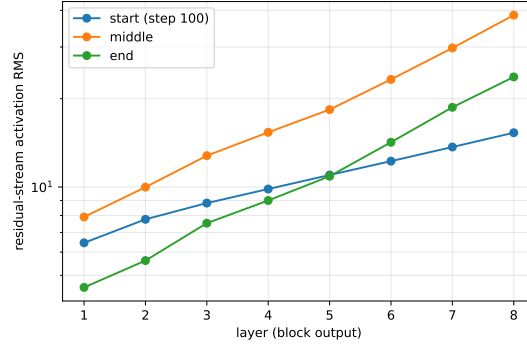


Figure 22: Default d8 run: RMS of the residual stream after each block, at the start, middle and end of training.

Figure 23 shows the same residual-stream RMS against the optimizer step for layers 1, 3 and 6, on the axes of the example question on page 18. In the healthy default run it stays between about 5 and 30 for the whole run, rising to a peak near step 1,800 and coming back down in the decay phase. Even the most disturbed run in this assignment, no warmup, only overshoots to a few hundred at step 100 before recovering. A residual-stream RMS that climbs to 10^6 is therefore not a healthy d8 at any hyperparameter setting I tried; activations at that scale saturate the softmax and the loss would be far above normal.

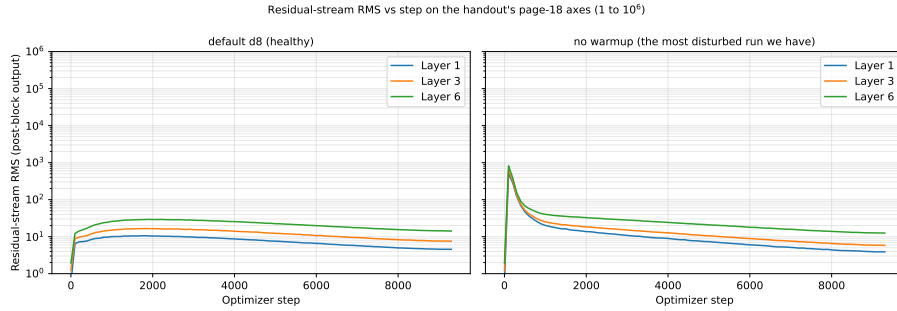


Figure 23: Residual-stream RMS of layers 1, 3 and 6 against optimizer step, on the axes of the page-18 example (log y from 1 to 10^6). Left: default d8. Right: the no-warmup run.

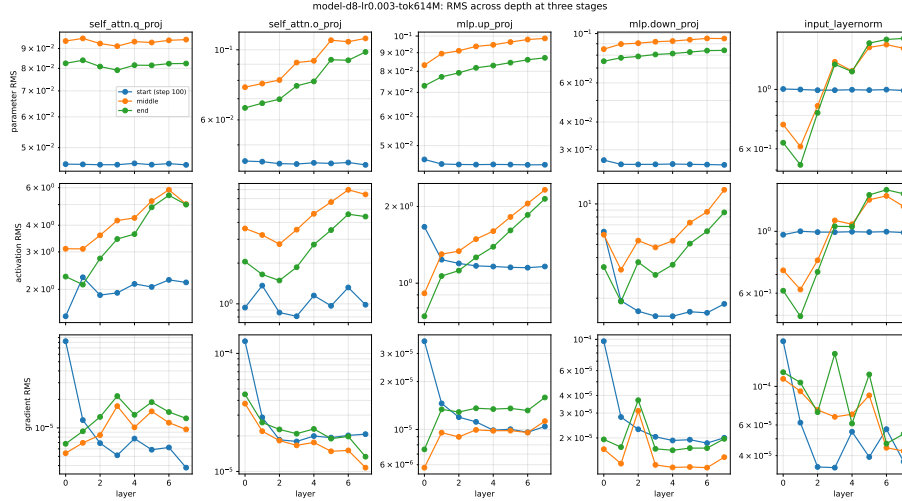


Figure 24: Default d8 run: RMS of parameters (top), activations (middle) and gradients (bottom) per layer for five modules, at the start, middle and end of training.

Takeaways. Learning rate and weight decay are the whole-run levers; warmup, clipping and tied embeddings are start-only levers; QK-norm moves the final activation scale without moving the weights.

7 Example question, page 14

The question asks whether batch 16 or lr 0.009 more than doubles the seed standard deviation of the final loss, over ten paired seeds each. My answer is neither (option 4). On my ten baseline seeds against three seeds per arm, batch 16 comes out at about $1.5\times$ and lr 0.009 at $2\text{--}3\times$, but three seeds cannot resolve a ratio below about $4\times$, and the ten-seed staff measurement puts lr 0.009 at $1.6\times$. Doubling the standard deviation takes a destabilising change such as removing warmup, which gave about $10\times$ (Problem 3(c)). My first answer was "both", from over-reading three-seed ratios; the lesson is that a standard deviation from three runs is uncertain by about half.